

2장. 재귀[자기호출]와 귀납적 사고



■ 주요 내용

- 01 자료구조와 재귀
- 02 재귀 구조 예
- 03 재귀와 수학적 귀납법

■ 학습목표

- 재귀의 뜻을 이해한다.
- 재귀가 사용되는 문제의 예들을 통해 기본적인 수준에서 재귀의 원리를 이해한다.
- 재귀와 수학적 귀납법 간의 밀접한 관계를 이해한다.

01 자료구조와 재귀

재귀란

- 내 안의 나를 찾는 것
- 성격은 같고 크기만 작은 나를 찾아 큰 나와 작은 내가 연결된 관계를 드러내는 것



그림 2-1 내 안에 나와 똑같지만 크기만 작은 것이 포함된 예

■ 재귀 알고리즘 = 자기호출 알고리즘

- 자신과 성격은 똑같지만 크기만 작은 알고리즘(들)을 호출하는 알고리즘
- 복잡한 문제도 간명하게 볼 수 있게 한다
- 예) 탐색, 정렬, 수열, ...
- 잘 쓰면 보약
 - 정렬, 탐색, ...
- 잘못 쓰면 독약
 - 피보나치수 구하기, 최적 행렬곱 경로, ...

02 재귀 구조 예

■ 초항이 1이고, 공차가 3인 등차수열

- n번째 원소는 자신과 성격이 똑같지만 순서가 하나 작은(n-1번째) 원소에 3을 더한 것

$$a_n = a_{n-1} + 3, a_1 = 1$$

알고리즘 2-1 등차수열

```
seq(n):  
    if (n = 1)  
        return 1  
    else  
        return seq(n-1) + 3
```

■ 재귀가 치명적인 예

$$f_n = f_{n-1} + f_{n-2}, f_1 = f_2 = 1$$

알고리즘 2-2 피보나치 수열(재귀 버전)

```
fib(n):  
    if (n = 1 or n = 2)  
        return 1  
    else  
        return fib(n-1) + fib(n-2)
```

← 재귀

Simple, but **terrible!**

알고리즘 2-3 피보나치 수열(비재귀 버전)

```
fib_fast(n):  
    f[1] ← f[2] ← 1 ◀ "f[2] ← 1"과 "f[1] ← 1"을 한꺼번에 적어놓은 것  
    for i ← 3 to n  
        f[i] ← f[i-1] + f[i-2]  
    return f[n]
```

← $\Theta(n)$

← 비재귀

재귀적 fib(100)은 얼마나 걸릴까?

내 데스크 탑 PC: Pentium 3GHz

fib(50) – 36초

fib(66) – 하루 정도

fib(100) – 3만5천년 정도

fib(136) – 1조년 초과

지수함수적 중복 호출로 인해 이런 치명적인 비효율이 발생한다

비재귀적 fib(100)은?

```
fib(n) :  
    t[1] ← t[2] ← 1  
    for i ← 3 to n  
        t[i] ← t[i-1] + t[i-2]  
    return t[n]
```

천만 분의 1초도 안 걸린다

$$n! = 1 \cdot 2 \cdot 3 \cdots n$$

$$\begin{aligned} n! &= n \cdot (n-1) \cdot (n-2) \cdots 1 \\ &= n \cdot (n-1)! \end{aligned} \quad \leftarrow \text{재귀적 구조}$$

$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * \text{factorial}(n-1) & \text{if } n > 0 \end{cases}$$

```
fact(n) :  
    tmp ← 1  
    for i ← 2 to n  
        tmp ← i * tmp  
    return tmp
```

← 비재귀

```
fact(n) :  
    if (n = 0) return 1  
    else return n * fact(n-1)
```

← 재귀

$\Theta(n)$

하노이 탑

- 원반 n 개, 기둥 3개 (a, b, c)
- 목표: n 개의 원반을 기둥 a로부터 기둥 b로 옮긴다

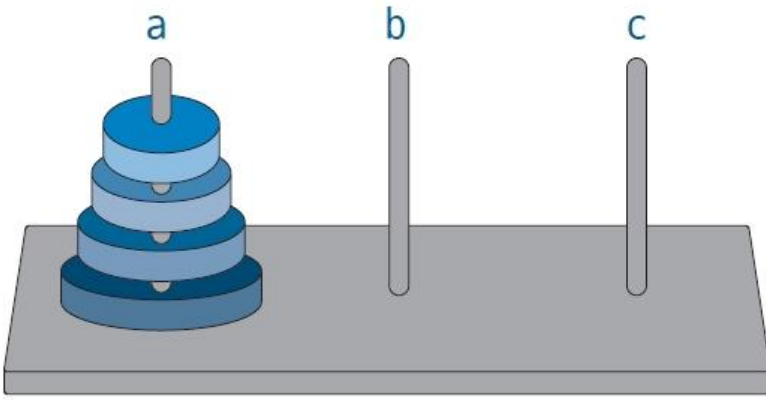


그림 2-2 원반이 4개인 하노이 탑 문제

- 원반은 한 번에 하나씩 옮길 수 있다.
- 큰 원반이 작은 원반 아래에 놓여 있어야 한다.

하노이 탑

- 함수 정의: `move(n, source, destination, spare)`
 - ◀ n 개의 원반을 `source` 기둥으로부터 `destination` 기둥으로 옮긴다.
 - ◀ Spare : 보조 기둥

- 예: `move(4, a, b, c)`

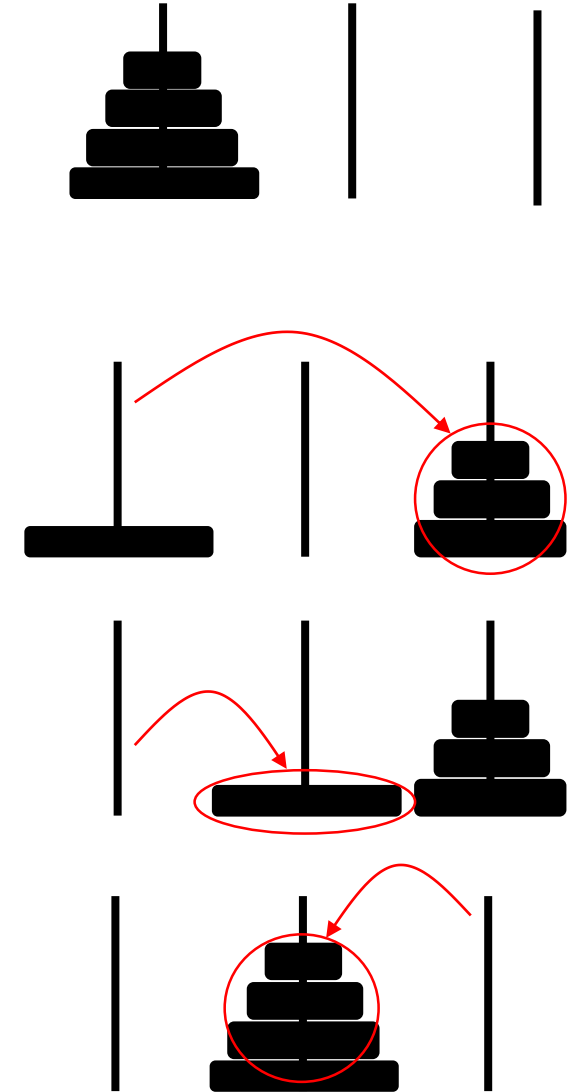
`move(3, a, c, b)`

a 에 있는 (유일한) 원반을 b 로 옮긴다

`move(3, c, b, a)`

- 목표 : `move(n, a, b, c)`

- [동영상으로 보기](#)



하노이 탑

알고리즘 2-4 하노이 탑

◀ 기둥 a에 있는 n개의 원반을 기둥 c를 보조 기둥으로 사용해 기둥 b로 옮긴다.

`move(n, a, b, c):`

- ① `if (n > 0)`
 - ② `move(n-1, a, c, b)`
 - ③ a에 있는 원반을 b로 옮긴다
 - ④ `move(n-1, c, b, a)`

↓ 마지막 직전 단계

`move(1, x, y, z):`

- ① `if (1 > 0)`
 - ② `move(0, x, z, y)`
x에 있는 원반을 y로 옮긴다
 - ④ `move(0, z, y, x)`

1. 최대 원소를 찾는다
2. 최대 원소와 맨 오른쪽 원소를 자리 바꾼다
3. 맨 오른쪽 자리를 관심 대상에서 제외한다
4. 원소가 1 개 남을 때까지 위 1~3의 순환을 반복

이 과정을 한번 끝내면 자신과 동일하지만 크기가 하나 작은 문제를 만난다



알고리즘 2-5 선택 정렬

`selectionSort(A[], n):` ◀ 배열 $A[0 \dots n-1]$ 을 정렬한다

- ① for $\text{last} \leftarrow n-1$ downto 1
- ② $A[0 \dots \text{last}]$ 중 가장 큰 수 $A[k]$ 를 찾는다
- ③ $A[k] \leftrightarrow A[\text{last}]$ ◀ $A[k]$ 와 $A[\text{last}]$ 의 값을 교환한다

↓ 이를 재귀 알고리즘으로 표현하면

알고리즘 2-6 선택 정렬(재귀 버전)

`selectionSort(A[], n):` ◀ 배열 $A[0 \dots n-1]$ 을 정렬한다

- ① if $(n > 1)$
 - ② $A[0 \dots \text{last}]$ 중 가장 큰 수 $A[k]$ 를 찾는다
 - ③ $A[k] \leftrightarrow A[\text{last}]$ ◀ $A[k]$ 와 $A[\text{last}]$ 의 값을 교환한다
 - ④ `selectionSort(A, n-1)` ◀ 배열 $A[0 \dots n-2]$ 를 정렬한다

중위, 전위, 후위 표현법

■ 수식을 표현할 때 연산자의 상대적 위치에 따라

- 전위 표현 Prefix Expression
- 중위 표현 Infix Expression
- 후위 표현 Postfix Expression

로 나뉜다

중위 표현법		전위 표현법	후위 표현법
$A + B * C - 2$	→	$- + A * B C 2$	$A B C * + 2 -$
$(A + B) * (C - 2)$	→	$* + A B - C 2$	$A B + C 2 - *$

그림 2-3 중위, 전위, 후위 표현법 예

중위, 전위, 후위 표현법

■ 형식 언어로 표현하면

(a) 중위 표현법

$\langle \text{infix} \rangle = \langle \text{변수} \rangle \mid \langle \text{infix} \rangle \langle \text{연산자} \rangle \langle \text{infix} \rangle$

$\langle \text{연산자} \rangle = + \mid - \mid * \mid /$

$\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$

(b) 전위 표현법

$\langle \text{prefix} \rangle = \langle \text{변수} \rangle \mid \langle \text{연산자} \rangle \langle \text{prefix} \rangle \langle \text{prefix} \rangle$

$\langle \text{연산자} \rangle = + \mid - \mid * \mid /$

$\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$

(c) 후위 표현법

$\langle \text{postfix} \rangle = \langle \text{변수} \rangle \mid \langle \text{postfix} \rangle \langle \text{postfix} \rangle \langle \text{연산자} \rangle$

$\langle \text{연산자} \rangle = + \mid - \mid * \mid /$

$\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$

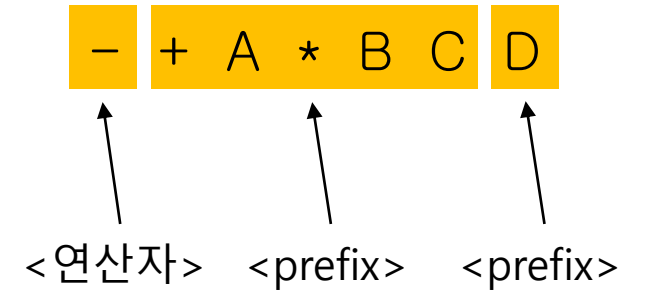


그림 2-4 재귀가 포함된 중위, 전위, 후위 표현법의 형식 언어

알고리즘 2-7 DFS 알고리즘

- ◀ 노드 x에서 이를 수 있는 모든 노드 찾기
- ◀ 모든 노드 v에 대해 v.visited는 false로 초기화됨

DFS(x):

① x.visited ← true

x에서 화살표로 연결된 모든 노드 y 각각에 대하여

② if (y.visited = false) DFS(y)

노드와 이들을 연결하는 간선으로 이루어진 그래프에서
시작 노드로부터 방문할 수 있는 모든 노드를 방문하는 작업

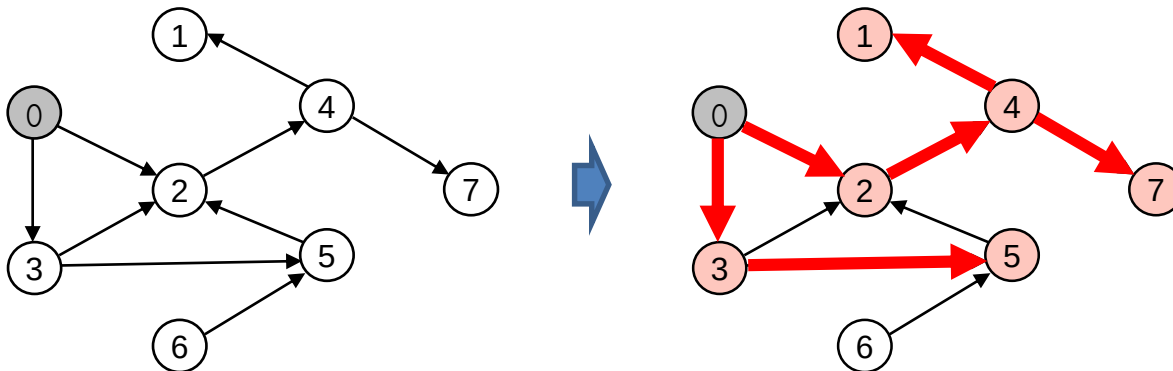


그림 2-5 깊이 우선 탐색의 예

0번 노드로부터
방문할 수 있는 노드들:
1, 2, 3, 4, 5, 7

깊이 우선 탐색 Depth-First Search

작동 예

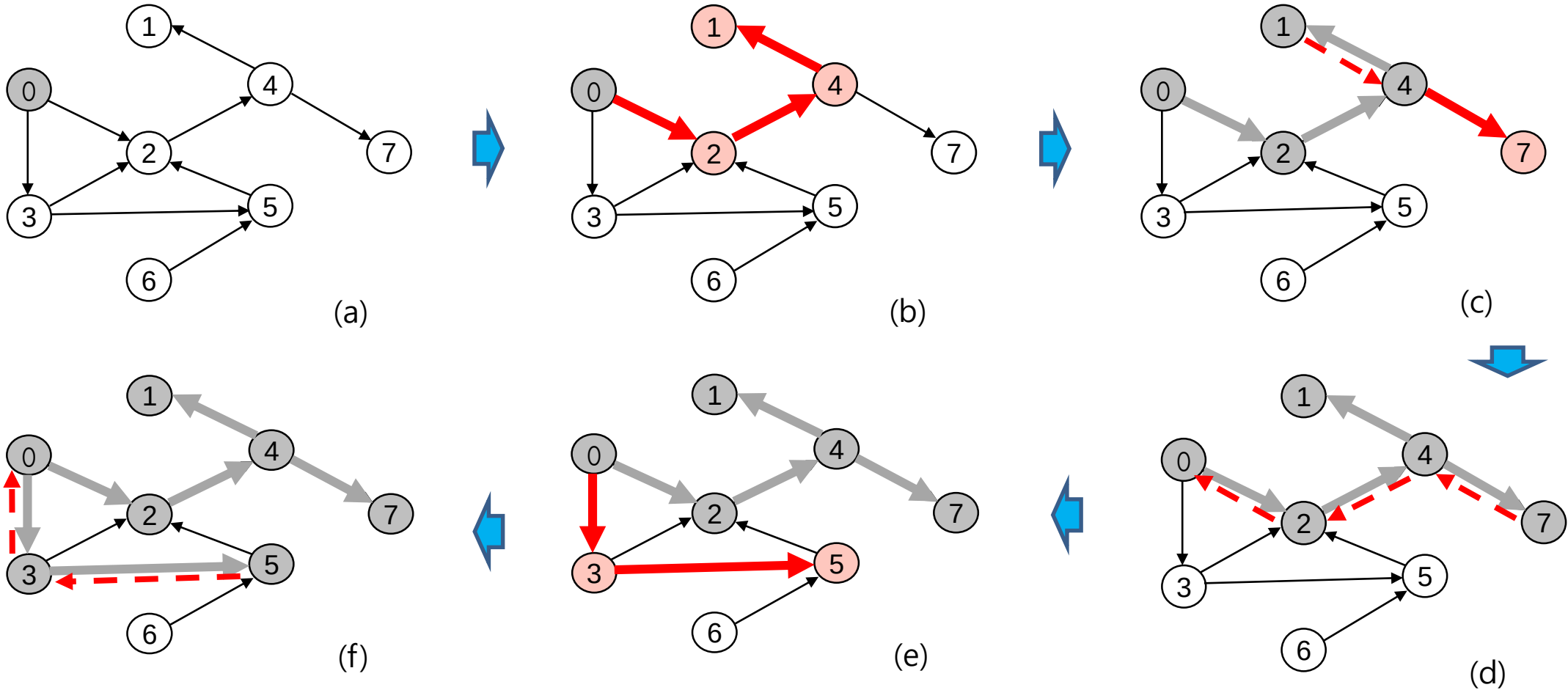


그림 2-6 DFS()의 수행 예

binarySearch ($A[], n, x$) :

◀ 정렬된 배열 $A[0 \dots n-1]$ 에서 원소 x 를 찾는다

$low \leftarrow 0$

$high \leftarrow n-1$

✓ Time: $O(\log n)$

while ($low \leq high$)

$mid \leftarrow (low + high)/2$

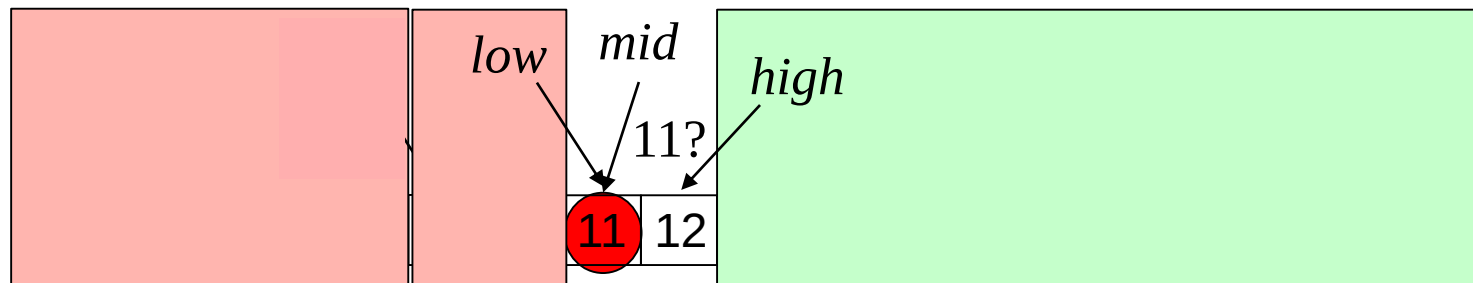
if ($A[mid] < x$) $low \leftarrow mid + 1$

else if ($A[mid] > x$) $high \leftarrow mid - 1$

else return mid

return "Not found"

$x = 11$ 을 찾아보자



✓ 재귀적 성격이 숨어있다. 그러면 아예 재귀 알고리즘으로 만들어보자.

✓ Time: $O(\log n)$

binarySearch(A[], x, low, high) :

◀ 정렬된 배열 A[low...high]에서 원소 x를 찾는다

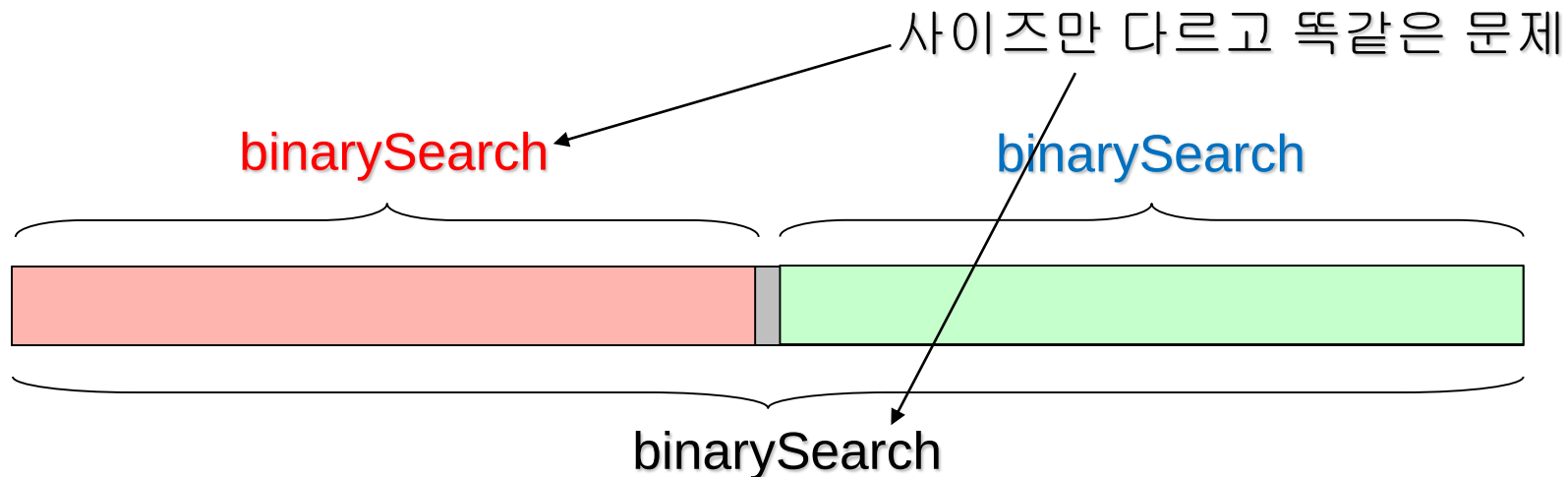
if (low > high) **return** "Not found"

mid ← (low + high)/2

if (A[mid] < x) **return** binarySearch(A, x, mid+1, high)

else if (A[mid] > x) **return** binarySearch(A, x, low, mid-1)

else return mid



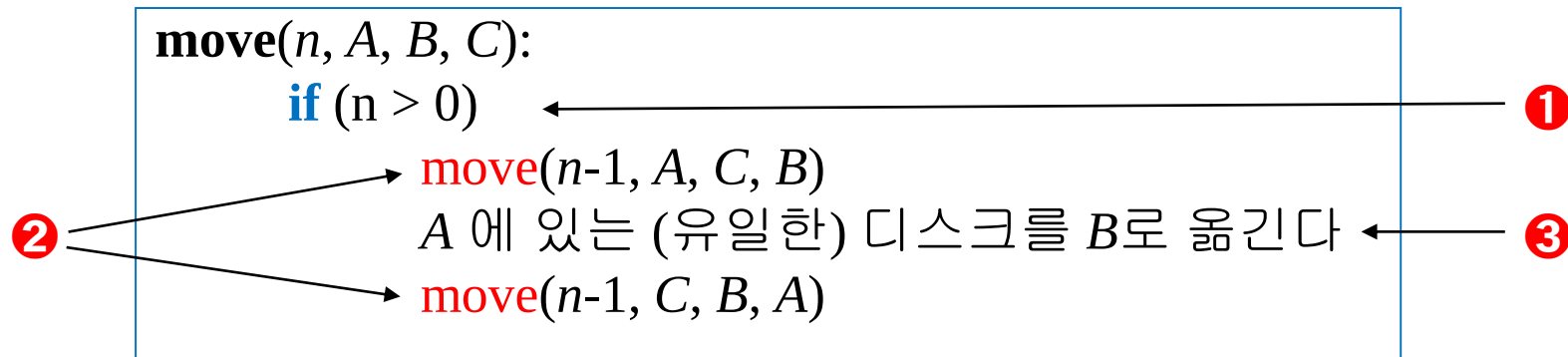
03 재귀와 수학적 귀납법

재귀 알고리즘의 필수 구비 조건

■ 재귀 알고리즘의 필수 구비 조건

- ① 경계 조건 **Base Condition** (또는 종료 조건): 재귀 호출이 반복되다 궁극적으로 끝나는 조건
- ② 재귀 호출
- ③ 관계: 닮음꼴 작은 문제(들)와 본 문제 간의 관계를 나타내는 부분

■ 하노이 탑의 예



경계(종료) 조건이 없으면?

Base case가 없으면?

fib(n) :

return (fib($n-1$)+fib($n-2$))

move(n , A , B , C) :

Base case가 없으면?

move($n-1$, A , C , B)

A 에 있는 (유일한) 디스크를 B 로 옮긴다

move($n-1$, C , B , A)

하노이 타워 문제와 수학적 귀납법

$T(n)$: 하노이 탑 알고리즘이 n 개의 원반을 옮기는 데 필요한 이동(원반 하나를 옮기는 것)의 총 횟수

$$T(n) = 2^n - 1$$

<증명: 수학적 귀납법>

- i) (경계조건 만족) 우선 0개를 옮기는 데는 이동 횟수가 0번이다. $T(0) = 2^0 - 1 = 0$ 이므로 만족한다.
- ii) (귀납적 가정) k 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k) = 2^k - 1$ 이라 가정하자.
- iii) (귀납적 전개: $k+1$ 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k+1) = 2^{k+1} - 1$ 이 됨을 보인다)
 $k+1$ 개의 원반을 옮기는 작업은 원반 k 개를 옮기는 작업 두 번, 원반 1개를 옮기는 작업 한 번
 으로 구성되므로 다음이 성립한다.

$$T(k+1) = 2T(k) + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$$

하노이 타워 문제와 수학적 귀납법

$T(n)$: 하노이 탑 알고리즘이 n 개의 원반을 옮기는 데 필요한 이동(원반 하나를 옮기는 것)의 총 횟수

Fact: $T(n) = 2^n - 1$

<증명>

경계 조건: $T(0) = 0 = 2^0 - 1$

귀납적 가정: $T(k) = 2^k - 1$ 라 가정하자

귀납적 전개:

$$\begin{aligned} T(k+1) &= 2 \cdot T(k) + 1 \\ &= 2(2^k - 1) + 1 \\ &= 2^{k+1} - 1 \end{aligned}$$



Thank You